

Security and Access Control

Internal identity, trust, secrets and privileged-control model.



This document explains its subject first in plain language, then in engineering detail. It is grounded in the Craves repository snapshot and deliberately separates current implementation, legacy/reference code, milestone foundations and repository gaps so readers do not mistake aspiration for proof.

Version 1.0 | Evidence date: 14 August 2026 | Repository: rmorampudi09-arch/Craves-Build-platform | Snapshot commit: 3225f7fa531a6b07185fb5e034f7930f8bd8b571

security layers - plain-language purpose

Plain-English explanation

Craves includes a governed semantic meal concierge for assisted discovery rather than an authority that can bypass commerce rules.

How it works technically

Recent main history and apps/customer-web-next/docs/signalr-ai-concierge.md describe SignalR semantic concierge work. Recommendations still rely on governed product data and ordinary catalog, quote, authorization and order paths for transactions.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The ai boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/docs/signalr-ai-concierge.md; commit 71e795b. Snapshot: main @ 3225f7fa531a (14 August 2026).

security layers - technical architecture

Plain-English explanation

Security is layered: identity proof, token validation, role/ownership checks, secret isolation, provider-signature checks and deployment gates address different failure modes.

How it works technically

APIM adds gateway policy, services re-check authorization, web uses secure cookies, mobile milestones use secure token storage, Razorpay callbacks use HMAC verification, and local starter secrets are prohibited in deployed Azure profiles where documented.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The security boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: [infra/apim/README.md](#); [services/user-chef-service/README.md](#); [services/order-service/README.md](#); [apps/customer-web-next/README.md](#). Snapshot: main @ 3225f7fa531a (14 August 2026).

security layers - end-to-end flow

Plain-English explanation

Security is layered: identity proof, token validation, role/ownership checks, secret isolation, provider-signature checks and deployment gates address different failure modes.

How it works technically

APIM adds gateway policy, services re-check authorization, web uses secure cookies, mobile milestones use secure token storage, Razorpay callbacks use HMAC verification, and local starter secrets are prohibited in deployed Azure profiles where documented.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The security boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: [infra/apim/README.md](#); [services/user-chef-service/README.md](#); [services/order-service/README.md](#); [apps/customer-web-next/README.md](#). Snapshot: main @ 3225f7fa531a (14 August 2026).

security layers - errors and edge cases

Plain-English explanation

Security is layered: identity proof, token validation, role/ownership checks, secret isolation, provider-signature checks and deployment gates address different failure modes.

How it works technically

APIM adds gateway policy, services re-check authorization, web uses secure cookies, mobile milestones use secure token storage, Razorpay callbacks use HMAC verification, and local starter secrets are prohibited in deployed Azure profiles where documented.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The security boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: [infra/apim/README.md](#); [services/user-chef-service/README.md](#); [services/order-service/README.md](#); [apps/customer-web-next/README.md](#). Snapshot: main @ 3225f7fa531a (14 August 2026).

security layers - deployment, security and operational validation

Plain-English explanation

The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls.

How it works technically

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The devops boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Snapshot: main @ 3225f7fa531a (14 August 2026).

threat boundaries - plain-language purpose

Plain-English explanation

Craves includes a governed semantic meal concierge for assisted discovery rather than an authority that can bypass commerce rules.

How it works technically

Recent main history and apps/customer-web-next/docs/signalr-ai-concierge.md describe SignalR semantic concierge work. Recommendations still rely on governed product data and ordinary catalog, quote, authorization and order paths for transactions.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The ai boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/docs/signalr-ai-concierge.md; commit 71e795b. Snapshot: main @ 3225f7fa531a (14 August 2026).

threat boundaries - technical architecture

Plain-English explanation

Security is layered: identity proof, token validation, role/ownership checks, secret isolation, provider-signature checks and deployment gates address different failure modes.

How it works technically

APIM adds gateway policy, services re-check authorization, web uses secure cookies, mobile milestones use secure token storage, Razorpay callbacks use HMAC verification, and local starter secrets are prohibited in deployed Azure profiles where documented.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The security boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: [infra/apim/README.md](#); [services/user-chef-service/README.md](#); [services/order-service/README.md](#); [apps/customer-web-next/README.md](#). Snapshot: main @ 3225f7fa531a (14 August 2026).

threat boundaries - end-to-end flow

Plain-English explanation

Security is layered: identity proof, token validation, role/ownership checks, secret isolation, provider-signature checks and deployment gates address different failure modes.

How it works technically

APIM adds gateway policy, services re-check authorization, web uses secure cookies, mobile milestones use secure token storage, Razorpay callbacks use HMAC verification, and local starter secrets are prohibited in deployed Azure profiles where documented.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The security boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: [infra/apim/README.md](#); [services/user-chef-service/README.md](#); [services/order-service/README.md](#); [apps/customer-web-next/README.md](#). Snapshot: main @ 3225f7fa531a (14 August 2026).

threat boundaries - errors and edge cases

Plain-English explanation

A useful failure model separates user input/state conflicts, identity/permission failures, dependency outages, data faults and deployment/configuration incidents.

How it works technically

order-service documents Problem Details fields and codes including ORDER_NOT_FOUND, ORDER_IDEMPOTENCY_CONFLICT, INVALID_STATUS, CANCELLATION_NOT_ALLOWED, REFUND_FAILED, UNAUTHORIZED, FORBIDDEN, BACKEND_CHANGE_DISABLED and BACKEND_UNAVAILABLE. Diagnostics should start from request/correlation evidence.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The errors boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: services/order-service/README.md; services/notification-service/README.md; apps/customer-web-next/README.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

threat boundaries - deployment, security and operational validation

Plain-English explanation

The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls.

How it works technically

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The devops boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Snapshot: main @ 3225f7fa531a (14 August 2026).

OTP identity - plain-language purpose

Plain-English explanation

Craves includes a governed semantic meal concierge for assisted discovery rather than an authority that can bypass commerce rules.

How it works technically

Recent main history and apps/customer-web-next/docs/signalr-ai-concierge.md describe SignalR semantic concierge work. Recommendations still rely on governed product data and ordinary catalog, quote, authorization and order paths for transactions.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The ai boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/docs/signalr-ai-concierge.md; commit 71e795b. Snapshot: main @ 3225f7fa531a (14 August 2026).

OTP identity - technical architecture

Plain-English explanation

Authentication proves who the caller is; authorization decides what that caller may do. Craves uses phone OTP for the current web and JWT-based authorization in backend services, with Entra External ID/PKCE documented for the mobile milestone.

How it works technically

Secure HTTP-only cookies keep web token state away from ordinary browser JavaScript. Backend services evaluate roles such as USER, CHEF and ADMIN and may also enforce object ownership. Mobile milestone guidance stores refresh credentials in Keychain/Keystore and access tokens in memory.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The auth boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; services/user-chef-service/README.md; services/order-service/README.md; docs/handover/2026-07-30-customer-mobile-auth-foundation.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

OTP identity - end-to-end flow

Plain-English explanation

Authentication proves who the caller is; authorization decides what that caller may do. Craves uses phone OTP for the current web and JWT-based authorization in backend services, with Entra External ID/PKCE documented for the mobile milestone.

How it works technically

Secure HTTP-only cookies keep web token state away from ordinary browser JavaScript. Backend services evaluate roles such as USER, CHEF and ADMIN and may also enforce object ownership. Mobile milestone guidance stores refresh credentials in Keychain/Keystore and access tokens in memory.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The auth boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; services/user-chef-service/README.md; services/order-service/README.md; docs/handover/2026-07-30-customer-mobile-auth-foundation.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

OTP identity - errors and edge cases

Plain-English explanation

Authentication proves who the caller is; authorization decides what that caller may do. Craves uses phone OTP for the current web and JWT-based authorization in backend services, with Entra External ID/PKCE documented for the mobile milestone.

How it works technically

Secure HTTP-only cookies keep web token state away from ordinary browser JavaScript. Backend services evaluate roles such as USER, CHEF and ADMIN and may also enforce object ownership. Mobile milestone guidance stores refresh credentials in Keychain/Keystore and access tokens in memory.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The auth boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; services/user-chef-service/README.md; services/order-service/README.md; docs/handover/2026-07-30-customer-mobile-auth-foundation.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

OTP identity - deployment, security and operational validation

Plain-English explanation

Authentication proves who the caller is; authorization decides what that caller may do. Craves uses phone OTP for the current web and JWT-based authorization in backend services, with Entra External ID/PKCE documented for the mobile milestone.

How it works technically

Secure HTTP-only cookies keep web token state away from ordinary browser JavaScript. Backend services evaluate roles such as USER, CHEF and ADMIN and may also enforce object ownership. Mobile milestone guidance stores refresh credentials in Keychain/Keystore and access tokens in memory.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The auth boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; services/user-chef-service/README.md; services/order-service/README.md; docs/handover/2026-07-30-customer-mobile-auth-foundation.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

JWT validation - plain-language purpose

Plain-English explanation

Craves includes a governed semantic meal concierge for assisted discovery rather than an authority that can bypass commerce rules.

How it works technically

Recent main history and [apps/customer-web-next/docs/signalr-ai-concierge.md](#) describe SignalR semantic concierge work. Recommendations still rely on governed product data and ordinary catalog, quote, authorization and order paths for transactions.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The ai boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: [apps/customer-web-next/docs/signalr-ai-concierge.md](#); commit 71e795b. Snapshot: main @ 3225f7fa531a (14 August 2026).

JWT validation - technical architecture

Plain-English explanation

Authentication proves who the caller is; authorization decides what that caller may do. Craves uses phone OTP for the current web and JWT-based authorization in backend services, with Entra External ID/PKCE documented for the mobile milestone.

How it works technically

Secure HTTP-only cookies keep web token state away from ordinary browser JavaScript. Backend services evaluate roles such as USER, CHEF and ADMIN and may also enforce object ownership. Mobile milestone guidance stores refresh credentials in Keychain/Keystore and access tokens in memory.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The auth boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; services/user-chef-service/README.md; services/order-service/README.md; docs/handover/2026-07-30-customer-mobile-auth-foundation.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

JWT validation - end-to-end flow

Plain-English explanation

Authentication proves who the caller is; authorization decides what that caller may do. Craves uses phone OTP for the current web and JWT-based authorization in backend services, with Entra External ID/PKCE documented for the mobile milestone.

How it works technically

Secure HTTP-only cookies keep web token state away from ordinary browser JavaScript. Backend services evaluate roles such as USER, CHEF and ADMIN and may also enforce object ownership. Mobile milestone guidance stores refresh credentials in Keychain/Keystore and access tokens in memory.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The auth boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; services/user-chef-service/README.md; services/order-service/README.md; docs/handover/2026-07-30-customer-mobile-auth-foundation.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

JWT validation - errors and edge cases

Plain-English explanation

Authentication proves who the caller is; authorization decides what that caller may do. Craves uses phone OTP for the current web and JWT-based authorization in backend services, with Entra External ID/PKCE documented for the mobile milestone.

How it works technically

Secure HTTP-only cookies keep web token state away from ordinary browser JavaScript. Backend services evaluate roles such as USER, CHEF and ADMIN and may also enforce object ownership. Mobile milestone guidance stores refresh credentials in Keychain/Keystore and access tokens in memory.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The auth boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; services/user-chef-service/README.md; services/order-service/README.md; docs/handover/2026-07-30-customer-mobile-auth-foundation.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

JWT validation - deployment, security and operational validation

Plain-English explanation

Authentication proves who the caller is; authorization decides what that caller may do. Craves uses phone OTP for the current web and JWT-based authorization in backend services, with Entra External ID/PKCE documented for the mobile milestone.

How it works technically

Secure HTTP-only cookies keep web token state away from ordinary browser JavaScript. Backend services evaluate roles such as USER, CHEF and ADMIN and may also enforce object ownership. Mobile milestone guidance stores refresh credentials in Keychain/Keystore and access tokens in memory.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The auth boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; services/user-chef-service/README.md; services/order-service/README.md; docs/handover/2026-07-30-customer-mobile-auth-foundation.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

RBAC - plain-language purpose

Plain-English explanation

Craves includes a governed semantic meal concierge for assisted discovery rather than an authority that can bypass commerce rules.

How it works technically

Recent main history and [apps/customer-web-next/docs/signalr-ai-concierge.md](#) describe SignalR semantic concierge work. Recommendations still rely on governed product data and ordinary catalog, quote, authorization and order paths for transactions.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The ai boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: [apps/customer-web-next/docs/signalr-ai-concierge.md](#); commit 71e795b. Snapshot: main @ 3225f7fa531a (14 August 2026).

RBAC - technical architecture

Plain-English explanation

Security is layered: identity proof, token validation, role/ownership checks, secret isolation, provider-signature checks and deployment gates address different failure modes.

How it works technically

APIM adds gateway policy, services re-check authorization, web uses secure cookies, mobile milestones use secure token storage, Razorpay callbacks use HMAC verification, and local starter secrets are prohibited in deployed Azure profiles where documented.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The security boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: [infra/apim/README.md](#); [services/user-chef-service/README.md](#); [services/order-service/README.md](#); [apps/customer-web-next/README.md](#). Snapshot: main @ 3225f7fa531a (14 August 2026).

RBAC - end-to-end flow

Plain-English explanation

Security is layered: identity proof, token validation, role/ownership checks, secret isolation, provider-signature checks and deployment gates address different failure modes.

How it works technically

APIM adds gateway policy, services re-check authorization, web uses secure cookies, mobile milestones use secure token storage, Razorpay callbacks use HMAC verification, and local starter secrets are prohibited in deployed Azure profiles where documented.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The security boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: [infra/apim/README.md](#); [services/user-chef-service/README.md](#); [services/order-service/README.md](#); [apps/customer-web-next/README.md](#). Snapshot: main @ 3225f7fa531a (14 August 2026).

RBAC - errors and edge cases

Plain-English explanation

A useful failure model separates user input/state conflicts, identity/permission failures, dependency outages, data faults and deployment/configuration incidents.

How it works technically

order-service documents Problem Details fields and codes including ORDER_NOT_FOUND, ORDER_IDEMPOTENCY_CONFLICT, INVALID_STATUS, CANCELLATION_NOT_ALLOWED, REFUND_FAILED, UNAUTHORIZED, FORBIDDEN, BACKEND_CHANGE_DISABLED and BACKEND_UNAVAILABLE. Diagnostics should start from request/correlation evidence.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The errors boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: services/order-service/README.md; services/notification-service/README.md; apps/customer-web-next/README.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

RBAC - deployment, security and operational validation

Plain-English explanation

The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls.

How it works technically

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The devops boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Snapshot: main @ 3225f7fa531a (14 August 2026).

ownership - plain-language purpose

Plain-English explanation

Craves includes a governed semantic meal concierge for assisted discovery rather than an authority that can bypass commerce rules.

How it works technically

Recent main history and apps/customer-web-next/docs/signalr-ai-concierge.md describe SignalR semantic concierge work. Recommendations still rely on governed product data and ordinary catalog, quote, authorization and order paths for transactions.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The ai boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/docs/signalr-ai-concierge.md; commit 71e795b. Snapshot: main @ 3225f7fa531a (14 August 2026).

ownership - technical architecture

Plain-English explanation

Security is layered: identity proof, token validation, role/ownership checks, secret isolation, provider-signature checks and deployment gates address different failure modes.

How it works technically

APIM adds gateway policy, services re-check authorization, web uses secure cookies, mobile milestones use secure token storage, Razorpay callbacks use HMAC verification, and local starter secrets are prohibited in deployed Azure profiles where documented.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The security boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: [infra/apim/README.md](#); [services/user-chef-service/README.md](#); [services/order-service/README.md](#); [apps/customer-web-next/README.md](#). Snapshot: main @ 3225f7fa531a (14 August 2026).

ownership - end-to-end flow

Plain-English explanation

Security is layered: identity proof, token validation, role/ownership checks, secret isolation, provider-signature checks and deployment gates address different failure modes.

How it works technically

APIM adds gateway policy, services re-check authorization, web uses secure cookies, mobile milestones use secure token storage, Razorpay callbacks use HMAC verification, and local starter secrets are prohibited in deployed Azure profiles where documented.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The security boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: [infra/apim/README.md](#); [services/user-chef-service/README.md](#); [services/order-service/README.md](#); [apps/customer-web-next/README.md](#). Snapshot: main @ 3225f7fa531a (14 August 2026).

ownership - errors and edge cases

Plain-English explanation

Security is layered: identity proof, token validation, role/ownership checks, secret isolation, provider-signature checks and deployment gates address different failure modes.

How it works technically

APIM adds gateway policy, services re-check authorization, web uses secure cookies, mobile milestones use secure token storage, Razorpay callbacks use HMAC verification, and local starter secrets are prohibited in deployed Azure profiles where documented.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The security boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: [infra/apim/README.md](#); [services/user-chef-service/README.md](#); [services/order-service/README.md](#); [apps/customer-web-next/README.md](#). Snapshot: main @ 3225f7fa531a (14 August 2026).

ownership - deployment, security and operational validation

Plain-English explanation

The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls.

How it works technically

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The devops boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Snapshot: main @ 3225f7fa531a (14 August 2026).

chef/admin isolation - plain-language purpose

Plain-English explanation

Chef mode is the supply side of the marketplace: onboarding, verification, menus, kitchen/order operations and earnings.

How it works technically

user-chef-service persists chef profiles and verification state. The frontend can switch context, but the server remains the authority: showing Chef mode never grants a CHEF role by itself.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The chef boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: services/user-chef-service/README.md; apps/customer-web-next/src/components/chef/; apps/customer-web-next/src/components/chef-mode/. Snapshot: main @ 3225f7fa531a (14 August 2026).

chef/admin isolation - technical architecture

Plain-English explanation

Chef mode is the supply side of the marketplace: onboarding, verification, menus, kitchen/order operations and earnings.

How it works technically

user-chef-service persists chef profiles and verification state. The frontend can switch context, but the server remains the authority: showing Chef mode never grants a CHEF role by itself.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The chef boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: services/user-chef-service/README.md; apps/customer-web-next/src/components/chef/; apps/customer-web-next/src/components/chef-mode/. Snapshot: main @ 3225f7fa531a (14 August 2026).

chef/admin isolation - end-to-end flow

Plain-English explanation

Chef mode is the supply side of the marketplace: onboarding, verification, menus, kitchen/order operations and earnings.

How it works technically

user-chef-service persists chef profiles and verification state. The frontend can switch context, but the server remains the authority: showing Chef mode never grants a CHEF role by itself.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The chef boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: services/user-chef-service/README.md; apps/customer-web-next/src/components/chef/; apps/customer-web-next/src/components/chef-mode/. Snapshot: main @ 3225f7fa531a (14 August 2026).

chef/admin isolation - errors and edge cases

Plain-English explanation

Chef mode is the supply side of the marketplace: onboarding, verification, menus, kitchen/order operations and earnings.

How it works technically

user-chef-service persists chef profiles and verification state. The frontend can switch context, but the server remains the authority: showing Chef mode never grants a CHEF role by itself.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The chef boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: services/user-chef-service/README.md; apps/customer-web-next/src/components/chef/; apps/customer-web-next/src/components/chef-mode/. Snapshot: main @ 3225f7fa531a (14 August 2026).

chef/admin isolation - deployment, security and operational validation

Plain-English explanation

Chef mode is the supply side of the marketplace: onboarding, verification, menus, kitchen/order operations and earnings.

How it works technically

user-chef-service persists chef profiles and verification state. The frontend can switch context, but the server remains the authority: showing Chef mode never grants a CHEF role by itself.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The chef boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: services/user-chef-service/README.md; apps/customer-web-next/src/components/chef/; apps/customer-web-next/src/components/chef-mode/. Snapshot: main @ 3225f7fa531a (14 August 2026).

HTTP-only cookies - plain-language purpose

Plain-English explanation

Craves includes a governed semantic meal concierge for assisted discovery rather than an authority that can bypass commerce rules.

How it works technically

Recent main history and apps/customer-web-next/docs/signalr-ai-concierge.md describe SignalR semantic concierge work. Recommendations still rely on governed product data and ordinary catalog, quote, authorization and order paths for transactions.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The ai boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/docs/signalr-ai-concierge.md; commit 71e795b. Snapshot: main @ 3225f7fa531a (14 August 2026).

HTTP-only cookies - technical architecture

Plain-English explanation

Security is layered: identity proof, token validation, role/ownership checks, secret isolation, provider-signature checks and deployment gates address different failure modes.

How it works technically

APIM adds gateway policy, services re-check authorization, web uses secure cookies, mobile milestones use secure token storage, Razorpay callbacks use HMAC verification, and local starter secrets are prohibited in deployed Azure profiles where documented.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The security boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: [infra/apim/README.md](#); [services/user-chef-service/README.md](#); [services/order-service/README.md](#); [apps/customer-web-next/README.md](#). Snapshot: main @ 3225f7fa531a (14 August 2026).

HTTP-only cookies - end-to-end flow

Plain-English explanation

Security is layered: identity proof, token validation, role/ownership checks, secret isolation, provider-signature checks and deployment gates address different failure modes.

How it works technically

APIM adds gateway policy, services re-check authorization, web uses secure cookies, mobile milestones use secure token storage, Razorpay callbacks use HMAC verification, and local starter secrets are prohibited in deployed Azure profiles where documented.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The security boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: [infra/apim/README.md](#); [services/user-chef-service/README.md](#); [services/order-service/README.md](#); [apps/customer-web-next/README.md](#). Snapshot: main @ 3225f7fa531a (14 August 2026).

HTTP-only cookies - errors and edge cases

Plain-English explanation

A useful failure model separates user input/state conflicts, identity/permission failures, dependency outages, data faults and deployment/configuration incidents.

How it works technically

order-service documents Problem Details fields and codes including ORDER_NOT_FOUND, ORDER_IDEMPOTENCY_CONFLICT, INVALID_STATUS, CANCELLATION_NOT_ALLOWED, REFUND_FAILED, UNAUTHORIZED, FORBIDDEN, BACKEND_CHANGE_DISABLED and BACKEND_UNAVAILABLE. Diagnostics should start from request/correlation evidence.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The errors boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: services/order-service/README.md; services/notification-service/README.md; apps/customer-web-next/README.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

HTTP-only cookies - deployment, security and operational validation

Plain-English explanation

The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls.

How it works technically

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The devops boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Snapshot: main @ 3225f7fa531a (14 August 2026).

mobile secure tokens - plain-language purpose

Plain-English explanation

Native mobile is proven mainly through milestone identity/API/CI contracts rather than a clearly complete runnable app on the reviewed main snapshot.

How it works technically

The 30 July handover specifies Entra External ID, authorization code with PKCE/MFA, refresh token in Keychain/Keystore, access token in memory, APIM bearer calls, one silent refresh after 401, then interactive sign-in. A short-lived React Native bootstrap branch was referenced.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The mobile boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Milestone foundation. Evidence: docs/handover/2026-07-30-customer-mobile-auth-foundation.md; docs/handover/2026-07-30-mobile-ci-foundation.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

mobile secure tokens - technical architecture

Plain-English explanation

Native mobile is proven mainly through milestone identity/API/CI contracts rather than a clearly complete runnable app on the reviewed main snapshot.

How it works technically

The 30 July handover specifies Entra External ID, authorization code with PKCE/MFA, refresh token in Keychain/Keystore, access token in memory, APIM bearer calls, one silent refresh after 401, then interactive sign-in. A short-lived React Native bootstrap branch was referenced.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The mobile boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Milestone foundation. Evidence: docs/handover/2026-07-30-customer-mobile-auth-foundation.md; docs/handover/2026-07-30-mobile-ci-foundation.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

mobile secure tokens - end-to-end flow

Plain-English explanation

Native mobile is proven mainly through milestone identity/API/CI contracts rather than a clearly complete runnable app on the reviewed main snapshot.

How it works technically

The 30 July handover specifies Entra External ID, authorization code with PKCE/MFA, refresh token in Keychain/Keystore, access token in memory, APIM bearer calls, one silent refresh after 401, then interactive sign-in. A short-lived React Native bootstrap branch was referenced.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The mobile boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Milestone foundation. Evidence: docs/handover/2026-07-30-customer-mobile-auth-foundation.md; docs/handover/2026-07-30-mobile-ci-foundation.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

mobile secure tokens - errors and edge cases

Plain-English explanation

Native mobile is proven mainly through milestone identity/API/CI contracts rather than a clearly complete runnable app on the reviewed main snapshot.

How it works technically

The 30 July handover specifies Entra External ID, authorization code with PKCE/MFA, refresh token in Keychain/Keystore, access token in memory, APIM bearer calls, one silent refresh after 401, then interactive sign-in. A short-lived React Native bootstrap branch was referenced.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The mobile boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Milestone foundation. Evidence: docs/handover/2026-07-30-customer-mobile-auth-foundation.md; docs/handover/2026-07-30-mobile-ci-foundation.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

mobile secure tokens - deployment, security and operational validation

Plain-English explanation

Native mobile is proven mainly through milestone identity/API/CI contracts rather than a clearly complete runnable app on the reviewed main snapshot.

How it works technically

The 30 July handover specifies Entra External ID, authorization code with PKCE/MFA, refresh token in Keychain/Keystore, access token in memory, APIM bearer calls, one silent refresh after 401, then interactive sign-in. A short-lived React Native bootstrap branch was referenced.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The mobile boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Milestone foundation. Evidence: docs/handover/2026-07-30-customer-mobile-auth-foundation.md; docs/handover/2026-07-30-mobile-ci-foundation.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

PKCE/MFA - plain-language purpose

Plain-English explanation

Native mobile is proven mainly through milestone identity/API/CI contracts rather than a clearly complete runnable app on the reviewed main snapshot.

How it works technically

The 30 July handover specifies Entra External ID, authorization code with PKCE/MFA, refresh token in Keychain/Keystore, access token in memory, APIM bearer calls, one silent refresh after 401, then interactive sign-in. A short-lived React Native bootstrap branch was referenced.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The mobile boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Milestone foundation. Evidence: docs/handover/2026-07-30-customer-mobile-auth-foundation.md; docs/handover/2026-07-30-mobile-ci-foundation.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

PKCE/MFA - technical architecture

Plain-English explanation

Native mobile is proven mainly through milestone identity/API/CI contracts rather than a clearly complete runnable app on the reviewed main snapshot.

How it works technically

The 30 July handover specifies Entra External ID, authorization code with PKCE/MFA, refresh token in Keychain/Keystore, access token in memory, APIM bearer calls, one silent refresh after 401, then interactive sign-in. A short-lived React Native bootstrap branch was referenced.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The mobile boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Milestone foundation. Evidence: docs/handover/2026-07-30-customer-mobile-auth-foundation.md; docs/handover/2026-07-30-mobile-ci-foundation.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

PKCE/MFA - end-to-end flow

Plain-English explanation

Native mobile is proven mainly through milestone identity/API/CI contracts rather than a clearly complete runnable app on the reviewed main snapshot.

How it works technically

The 30 July handover specifies Entra External ID, authorization code with PKCE/MFA, refresh token in Keychain/Keystore, access token in memory, APIM bearer calls, one silent refresh after 401, then interactive sign-in. A short-lived React Native bootstrap branch was referenced.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The mobile boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Milestone foundation. Evidence: docs/handover/2026-07-30-customer-mobile-auth-foundation.md; docs/handover/2026-07-30-mobile-ci-foundation.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

PKCE/MFA - errors and edge cases

Plain-English explanation

Native mobile is proven mainly through milestone identity/API/CI contracts rather than a clearly complete runnable app on the reviewed main snapshot.

How it works technically

The 30 July handover specifies Entra External ID, authorization code with PKCE/MFA, refresh token in Keychain/Keystore, access token in memory, APIM bearer calls, one silent refresh after 401, then interactive sign-in. A short-lived React Native bootstrap branch was referenced.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The mobile boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Milestone foundation. Evidence: docs/handover/2026-07-30-customer-mobile-auth-foundation.md; docs/handover/2026-07-30-mobile-ci-foundation.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

PKCE/MFA - deployment, security and operational validation

Plain-English explanation

Native mobile is proven mainly through milestone identity/API/CI contracts rather than a clearly complete runnable app on the reviewed main snapshot.

How it works technically

The 30 July handover specifies Entra External ID, authorization code with PKCE/MFA, refresh token in Keychain/Keystore, access token in memory, APIM bearer calls, one silent refresh after 401, then interactive sign-in. A short-lived React Native bootstrap branch was referenced.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The mobile boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Milestone foundation. Evidence: docs/handover/2026-07-30-customer-mobile-auth-foundation.md; docs/handover/2026-07-30-mobile-ci-foundation.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

secret storage - plain-language purpose

Plain-English explanation

Craves includes a governed semantic meal concierge for assisted discovery rather than an authority that can bypass commerce rules.

How it works technically

Recent main history and apps/customer-web-next/docs/signalr-ai-concierge.md describe SignalR semantic concierge work. Recommendations still rely on governed product data and ordinary catalog, quote, authorization and order paths for transactions.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The ai boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/docs/signalr-ai-concierge.md; commit 71e795b. Snapshot: main @ 3225f7fa531a (14 August 2026).

secret storage - technical architecture

Plain-English explanation

Security is layered: identity proof, token validation, role/ownership checks, secret isolation, provider-signature checks and deployment gates address different failure modes.

How it works technically

APIM adds gateway policy, services re-check authorization, web uses secure cookies, mobile milestones use secure token storage, Razorpay callbacks use HMAC verification, and local starter secrets are prohibited in deployed Azure profiles where documented.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The security boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: [infra/apim/README.md](#); [services/user-chef-service/README.md](#); [services/order-service/README.md](#); [apps/customer-web-next/README.md](#). Snapshot: main @ 3225f7fa531a (14 August 2026).

secret storage - end-to-end flow

Plain-English explanation

Security is layered: identity proof, token validation, role/ownership checks, secret isolation, provider-signature checks and deployment gates address different failure modes.

How it works technically

APIM adds gateway policy, services re-check authorization, web uses secure cookies, mobile milestones use secure token storage, Razorpay callbacks use HMAC verification, and local starter secrets are prohibited in deployed Azure profiles where documented.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The security boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: [infra/apim/README.md](#); [services/user-chef-service/README.md](#); [services/order-service/README.md](#); [apps/customer-web-next/README.md](#). Snapshot: main @ 3225f7fa531a (14 August 2026).

secret storage - errors and edge cases

Plain-English explanation

Security is layered: identity proof, token validation, role/ownership checks, secret isolation, provider-signature checks and deployment gates address different failure modes.

How it works technically

APIM adds gateway policy, services re-check authorization, web uses secure cookies, mobile milestones use secure token storage, Razorpay callbacks use HMAC verification, and local starter secrets are prohibited in deployed Azure profiles where documented.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The security boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: [infra/apim/README.md](#); [services/user-chef-service/README.md](#); [services/order-service/README.md](#); [apps/customer-web-next/README.md](#). Snapshot: main @ 3225f7fa531a (14 August 2026).

secret storage - deployment, security and operational validation

Plain-English explanation

The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls.

How it works technically

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The devops boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Snapshot: main @ 3225f7fa531a (14 August 2026).